ALGORITHMS · PRINTABLE WORKBOOK

How to Walk a Graph

Graphs, breadth-first and depth-first search, the two-colour test and topological order (COMPSCI 311, lectures 4 to 6), on paper: worked examples on the lectures' own graphs, warm-ups, and 32 questions in four tiers, with an answer key worked out by the same code the website runs.

How to use this packet

1. Work in order. Each part starts with a **Learn** page and a worked example, then a warm-up you do yourself, then the questions.
2. Where a run depends on the order neighbours are looked at, the packet says the order. Otherwise any order gives the same answer.
3. Questions marked **clicker** are the lecture's own, with their original wording.
4. The **answer key** is at the back. Where more than one answer is right (a topological order, say), the key says how to check yours instead of asking you to match.

Part 1	The expanding wave	9 questions
Part 2	Queue or stack	8 questions
Part 3	Two colours	6 questions
Part 4	All edges forward	9 questions
Summary	One page to keep	
Key	Warm-ups and every question	

PART 1 · LEARN

The expanding wave

A **graph** G = (V, E) is a set of nodes and a set of edges, each edge a pair of nodes. We write n = |V| and m = |E|. Unless it says otherwise, edges have no direction.

Words for moving around a graph

- A **path** is a sequence of nodes where each one after the first is joined by an edge to the one before. Nodes may repeat.
- A **simple path** repeats no node. A **cycle** ends where it starts, repeats no other node and no edge, and goes round at least three nodes.
- The **distance** from u to v is the fewest edges on any path from u to v.
- A graph is **connected** if there is a path between every pair of nodes. A **connected component** is a largest set of nodes with a path between every pair.
- **In CLRS** (Appendix B.4), path, simple path and connected component mean the same. What is called a cycle here, CLRS calls a **simple cycle**: its plain "cycle" may pass through a node twice. CLRS writes V and E for the sets of nodes and edges, so O(m + n) is its O(V + E).

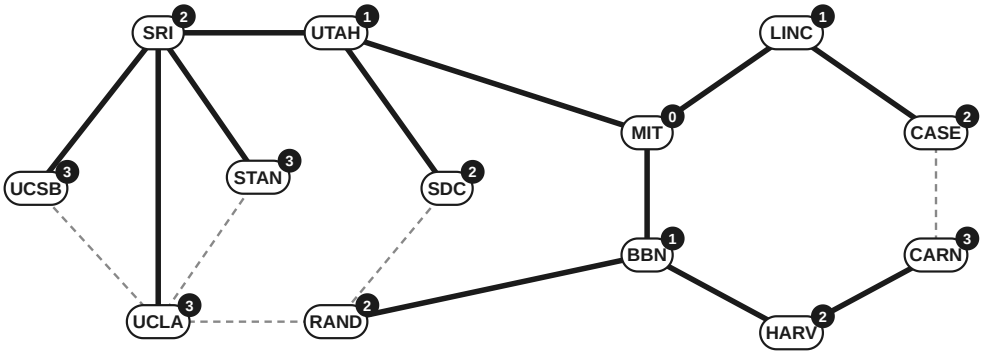
Breadth-first search, layer by layer

```
BFS(s)
  mark s discovered; L[0] = {s}; i = 0
  while L[i] is not empty
    L[i+1] = empty list
    for each node v in L[i]
      for each edge (v, w)
        if w is not discovered
          mark w; add w to L[i+1]; add (v, w) to T
    i = i + 1
```

Layer i is exactly the nodes at distance i from s. So there is a path from s to t exactly when t is in some layer. The order you look at neighbours in can change which edges end up in the tree, but never the layers. (CLRS §22.2 runs BFS with a queue and stores the layer of v as v.d. Its white, gray and black mean undiscovered, discovered and explored.)

Worked example: the Internet in 1970, from MIT

Neighbours are looked at in this order throughout: MIT, UTAH, BBN, LINC, SRI, SDC, RAND, HARV, CASE, UCSB, STAN, UCLA, CARN.



Badges: layer. Solid: BFS tree edges. Dashed: edges BFS looked at and did not need.

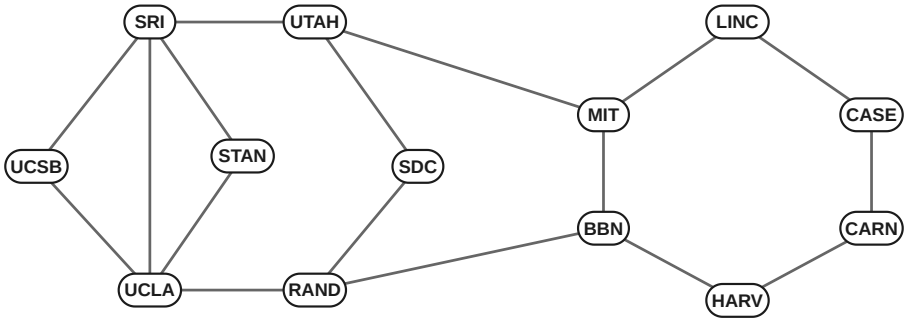
L_0	L_1	L_2	L_3
MIT	UTAH BBN LINC	SRI SDC RAND HARV CASE	UCSB STAN UCLA CARN

Five edges are dashed: SDC–RAND, UCSB–UCLA and STAN–UCLA join two nodes in the *same* layer, and RAND–UCLA and CASE–CARN join neighbouring layers. None skips a layer, and none ever can: when x is explored, every undiscovered neighbour of x goes into the next layer.

PART 1 · TRY IT

Run it yourself

WARM-UP W1 · THE SAME GRAPH, FROM SRI



node	MIT	UTAH	BBN	LINC	SRI	SDC	RAND	HARV	CASE
layer					0				

node	UCSB	STAN	UCLA	CARN
layer				

Now draw the BFS tree on the picture: for each node, the edge it was first found along. Which edges are left over?

WARM-UP W1B · WHAT IT COSTS

For the worked example (BFS from MIT), how many times does each line run? Use n = 13 and m = 17.

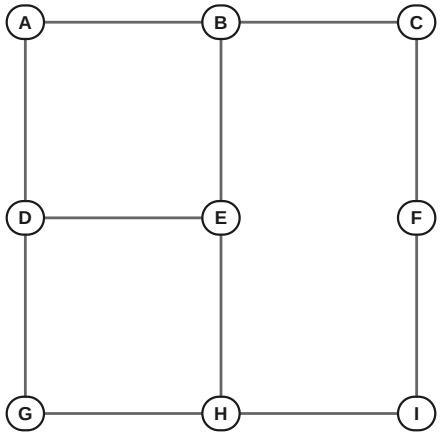
line	runs
`mark s discovered; L[0] = {s}; i = 0`	
`while L[i] is not empty`	
`L[i+1] = empty list`	
`for each node v in L[i]`	
`for each edge (v, w)`	
`if w is not discovered`	
`mark w; add w to L[i+1]; add (v, w) to T`	
`i = i + 1`	

Which lines run 2m times, and why twice?

Tier 1: The expanding wave

1.1 *Skill: running BFS by hand on a graph the page never animated*

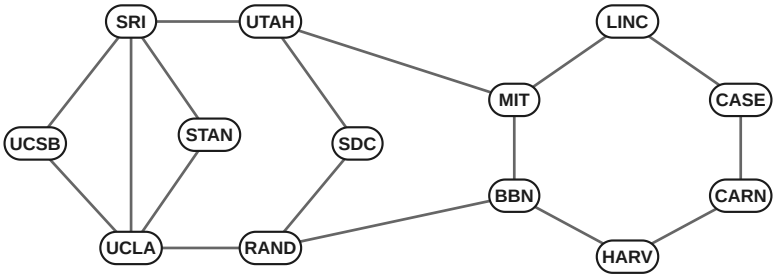
Run BFS from A. Put every corner in its layer.



node	A	B	C	D	E	F	G	H	I
layer	0								

1.2 *Skill: reading layers off a real graph*

Clicker · Lecture 4, slide 22



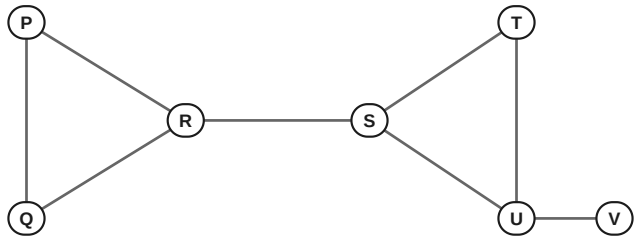
Run BFS on the 1970 Internet starting from MIT. How many nodes are in layer 2?

Nodes in layer 2 =

Working:

1.3 *Skill: the vocabulary: path, simple path, cycle*

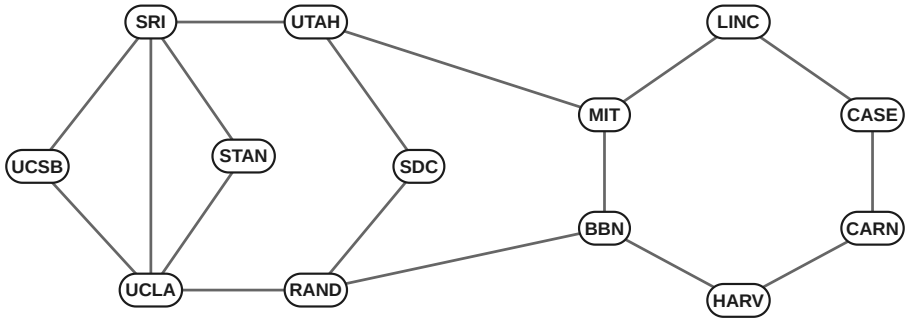
Name each sequence of nodes.



sequence	Not a path	Path, not simple	Simple path	Cycle
P – Q – R – S	◯	◯	◯	◯
S – T – U – S	◯	◯	◯	◯
R – S – R – P	◯	◯	◯	◯
Q – R – S – U – V	◯	◯	◯	◯
P – R – T	◯	◯	◯	◯
P – Q – R – P	◯	◯	◯	◯
U – S – T – U – V	◯	◯	◯	◯

1.4 *Skill: which edges the BFS tree does not use*

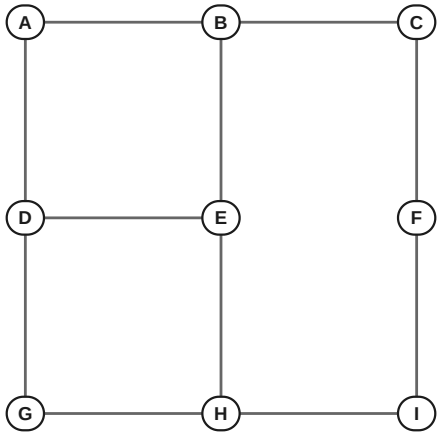
Run BFS on the 1970 Internet from SRI, looking at each node's neighbours in the order MIT, UTAH, BBN, LINC, SRI, SDC, RAND, HARV, CASE, UCSB, STAN, UCLA, CARN. Tick every edge that is not in the BFS tree.



Tick each edge that belongs.

☐ MIT–UTAH ☐ MIT–BBN ☐ MIT–LINC ☐ UTAH–SRI ☐ UTAH–SDC ☐ BBN–RAND ☐ BBN–HARV
☐ LINC–CASE ☐ SDC–RAND ☐ SRI–UCSB ☐ SRI–STAN ☐ SRI–UCLA ☐ RAND–UCLA ☐ HARV–CARN
☐ CASE–CARN ☐ STAN–UCLA ☐ UCSB–UCLA

1.5 *Skill: counting how often a line runs, the running-time argument*



Run BFS on the small town from A. How many times does the line "if w is not discovered" run?

Times it runs = ☐

Working:

1.6 *Skill: what the O(m + n) bound assumes about how the graph is stored*

Store the graph as an adjacency matrix instead of adjacency lists: an n × n table with a 1 in row v, column w when (v, w) is an edge. Now BFS can only find the neighbours of v by reading all of row v. What is its running time?

- ◯ **A.** O(m + n), as before.
- ◯ **B.** $O(n^2)$
- ◯ **C.** O(nm)
- ◯ **D.** O(n)

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.7 *Skill: what the search order can and cannot change*

Clicker · Lecture 4, slide 30

Suppose BFS explores the nodes within each layer in a different order. What can change?

- ◯ **A.** The layers
- ◯ **B.** The BFS tree
- ◯ **C.** Both
- ◯ **D.** Neither

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.8 *Skill: the proof that an edge spans at most one layer*

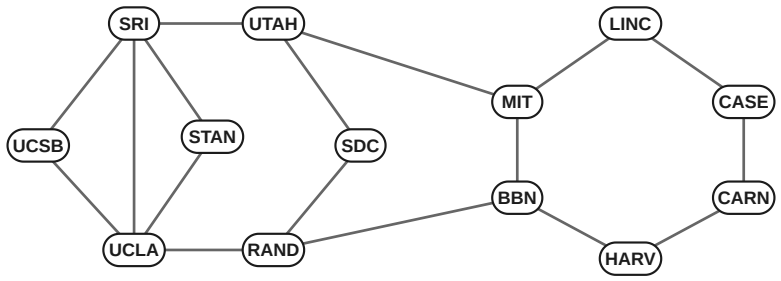
Edge (x, y), with x discovered first, in layer i. Why can y not end up in layer i + 2 or later?

- ◯ **A.** When x is explored, every undiscovered neighbour of x goes into layer i + 1, and y is a neighbour of x.
- ◯ **B.** Because BFS explores layer i before layer i + 1.
- ◯ **C.** Because the graph is connected.
- ◯ **D.** Because y was discovered first.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.9 *Skill: connectivity: what deleting edges and nodes does*

Clicker · Lecture 4, slide 16



Which statement about the 1970 Internet is false?

- ◯ **A.** Deleting any one edge of the graph must keep it connected.
- ◯ **B.** Deleting any two edges of the graph must disconnect it.
- ◯ **C.** Deleting any one node (with its edges) keeps it connected.
- ◯ **D.** There is a way to delete two nodes (with their edges) and disconnect it, but there is another way to keep it connected.

Why not the others? Pick the one you almost chose and say what is wrong with it.

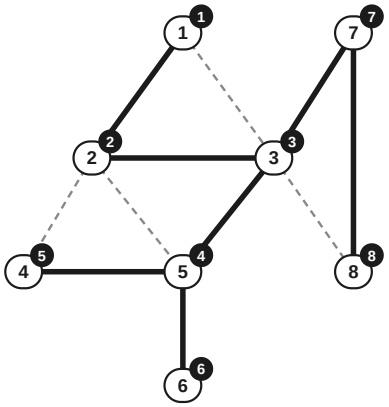
Queue or stack

Depth-first search

```
DFS(u)
  mark u explored
  for each edge (u, v)
    if v is not explored
      add (u, v) to T
      DFS(v)
```

DFS goes as deep as it can before backing up: a call only returns when every edge out of its node has been looked at. It is O(m + n) over all the calls together. CLRS (§22.3) also stamps each node with u.d when its call starts and u.f when it returns, on one clock that runs from 1 to 2n.

Worked example: the 8-node graph, from 1



Badges: the order DFS explores nodes in (neighbours in increasing order). Solid: DFS tree. Dashed: non-tree edges.

Order: 1, 2, 3, 5, 4, 6, 7, 8. The non-tree edges are 1–3, 2–4, 2–5, 3–8, and every one joins a node to one of its **ancestors**. That always happens: if x is explored first, DFS(x) looks along the edge (x, y) before it returns, so y is found inside DFS(x) and ends up below x.

One algorithm, one word apart

```
put s in A
while A is not empty
  take a node v from A
  if v is not marked explored
    mark v explored
    for each edge (v, w)
      put w in A
```

If A is a **queue** (first in, first out) this is BFS. If A is a **stack** (last in, first out) it is DFS. Either way it explores exactly what s can reach. A node is marked when it comes *out* of A. CLRS's BFS marks a node as it goes in, which is fine for a queue, but with a stack that rule builds a tree that is not depth first.

Every component

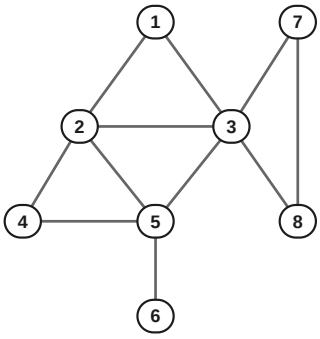
While some node is unexplored, start a search there. Each search finds one connected component, and the total is still O(m + n), because each search only pays for its own piece.

PART 2 · TRY IT

Run the one-word difference

WARM-UP W2 · QUEUE, THEN STACK

Run the generic traversal on the 8-node graph from 1, putting neighbours into A in increasing order. Do it once with A a queue and once with A a stack. Write down the order nodes are **explored** in, skipping any copy of a node that is already explored.



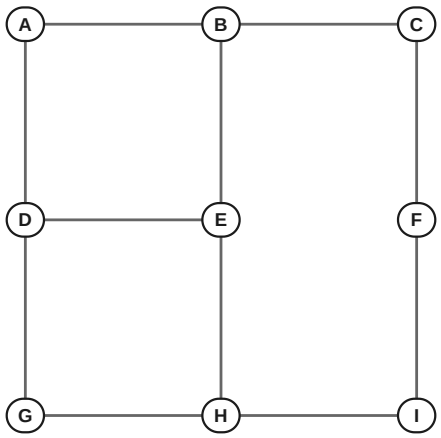
A is a	1st	2nd	3rd	4th	5th	6th	7th	8th
queue								
stack								

Count how many times node 5 went into A. Compare with its degree.

Tier 2: Queue or stack

2.1 *Skill: running recursive DFS by hand*

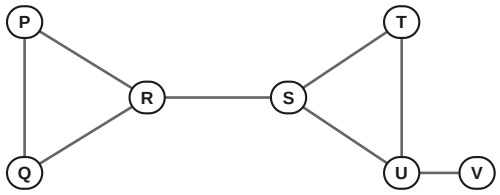
Run recursive DFS on the small town from A, looking at neighbours in alphabetical order. Tick every edge in the DFS tree.



Tick each edge that belongs.

☐ A–B ☐ A–D ☐ B–C ☐ B–E ☐ C–F ☐ D–E ☐ D–G ☐ E–H ☐ F–I ☐ H–I ☐ G–H

2.2 *Skill: CLRS's discovery and finishing times*



CLRS runs DFS with a clock that ticks once when a call starts and once when it returns. Node u is stamped u.d when DFS(u) starts and u.f when it returns, so the stamps run from 1 to 2n. Run recursive DFS on the two triangles from R, neighbours in alphabetical order. What is S.d?

S.d = ____

Working:

2.3 *Skill: the degree sum, which the running time rests on*

Clicker · Lecture 5, slide 10

Let q be the sum of the degrees of all the nodes in the graph. Which one of the following is false?

- ◯ **A.** q is twice the number of edges
- ◯ **B.** q is n times the average degree
- ◯ **C.** q is $\Theta(m + n)$ if $m \geq n$
- ◯ **D.** None of the above

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.4 *Skill: reading the generic traversal exactly*

Clicker · Lecture 5, slide 23

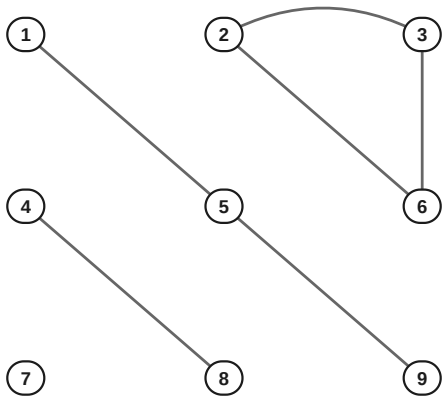
```
put s in A
while A is not empty
  take a node v from A
  if v is not marked "explored"
    mark v "explored"
    for each edge (v, w) incident to v
      put w in A
```

Suppose we run the traversal code and every node is marked explored before it terminates. Which of the following is false?

- ◯ **A.** Every node is marked "explored" exactly once.
- ◯ **B.** A single node could be put into A more than once.
- ◯ **C.** If $w \neq s$, the number of times that node w is put into A is degree(w).
- ◯ **D.** It is possible that there exist nodes x and y with no path from x to y.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.5 *Skill: connected components, and how many searches finding them takes*



The loop "while some node is unexplored, run a search from it" runs on this graph. How many searches does it start?

Searches started = ☐

Working:

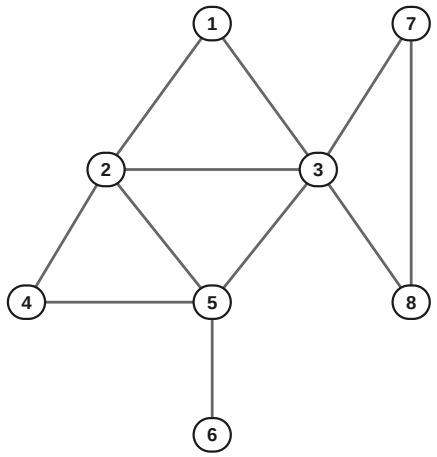
2.6 *Skill: what the one-word difference between BFS and DFS changes*

In the generic traversal, A is changed from a queue to a stack. What changes?

- ◯ **A.** Which nodes get explored.
- ◯ **B.** The order nodes are explored in, and so the tree.
- ◯ **C.** The running time, from $O(m + n)$ to $O(n^2)$.
- ◯ **D.** Nothing, since the pseudocode is the same.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.7 *Skill: why the stack version marks a node when it comes out, not when it goes in*




```
mark s; put s in A
while A is not empty
  take a node v from A
  for each edge (v, w)
    if w is not marked
      mark w; put w in A; add (v, w) to T
```

A common rewrite of the traversal marks w as soon as it is put in A, and never puts in a node that is already marked, so nothing goes in twice. With A a queue that is still BFS. Make A a stack and run it on the 8-node graph from 1, neighbours in increasing order. Is the tree it builds a depth-first tree?

◯ **A.** Yes. It is the page's stack version with the duplicates left out.

◯ **B.** No, because it explores different nodes.

◯ **C.** No. Taking 1 marks both 2 and 3 and puts them in, so both hang from 1, and the edge 2–3 then joins two nodes neither of which is an ancestor of the other.

◯ **D.** Yes, since any search that uses a stack is depth first.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.8 *Skill: the DFS non-tree edge claim*

In a DFS tree of an undirected graph, what can a non-tree edge (x, y) join?

- ◯ **A.** A node and one of its ancestors in the tree.
- ◯ **B.** Two nodes in different branches of the tree.
- ◯ **C.** Two nodes at the same depth.
- ◯ **D.** Anything. Non-tree edges have no structure.

Why not the others? Pick the one you almost chose and say what is wrong with it.

Two colours

A graph is **bipartite** if its nodes can be split into two sides, X and Y, with every edge joining X to Y. Colour one side blue and the other orange: no edge joins two nodes of the same colour. Students and colleges are the standard example; every application joins one of each. (CLRS Appendix B.4 defines it the same way.)

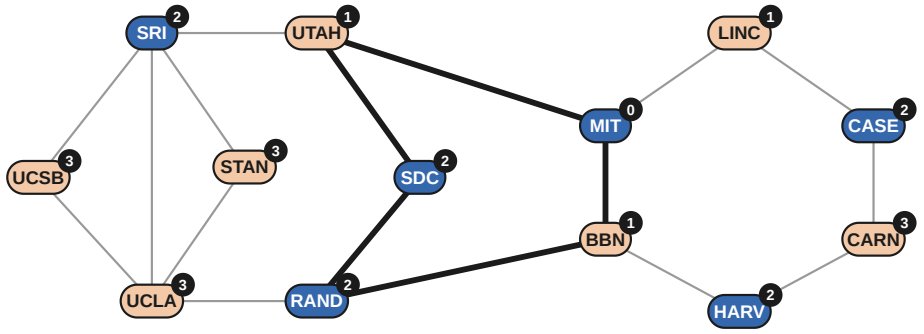
The test

1. Run BFS from any node.
2. Colour even layers blue and odd layers orange.
3. Look at every edge. If one joins two nodes in the same layer, say "not bipartite". Otherwise say "bipartite", with X the even layers and Y the odd ones.

Why it is right. BFS never lets an edge skip a layer. So an edge joins neighbouring layers (different colours, fine) or one layer. If the answer is "bipartite", every edge is the first kind. If it is "not bipartite", take the edge (x, y) inside layer j and walk x and y up the BFS tree until they meet at z, in layer i. The two tree paths plus the edge are a cycle of length 2(j − i) + 1: odd. And an odd cycle cannot be two-coloured, since colours alternate all the way round.

So: a graph is bipartite exactly when it has no odd cycle.

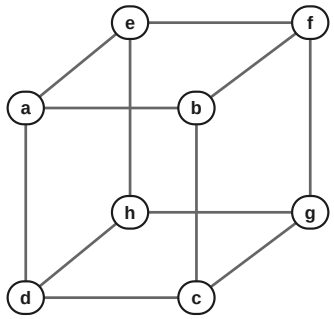
Worked example: the Internet in 1970



Dark: even layers. Light: odd layers. Heavy: the odd cycle.

SDC and RAND are both in layer 2. They meet going up at z = MIT, in layer 0: the cycle MIT → UTAH → SDC → RAND → BBN → MIT has 2(2 − 0) + 1 = 5 edges. Not bipartite.

WARM-UP W3 · THE CUBE

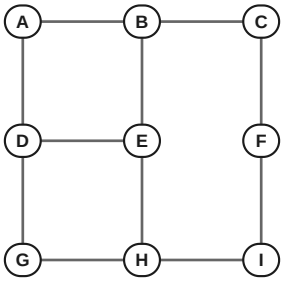


Run the test from a. Write each node's layer beside it and colour it. Is the cube bipartite? If yes, list the two sides; if no, give an odd cycle.

Tier 3: Two colours

3.1 *Skill: spotting bipartite graphs, or the odd cycle that rules one out*

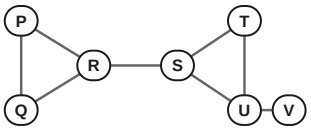
Which of these graphs are bipartite? Answer for each one.



A. A small town

☐ bipartite ☐ not

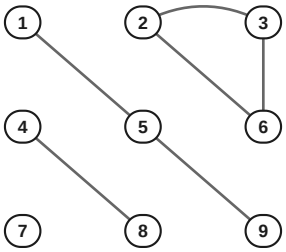
Evidence (a colouring, or an odd cycle):



B. Two triangles

☐ bipartite ☐ not

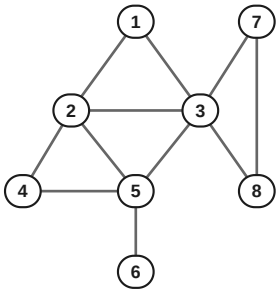
Evidence (a colouring, or an odd cycle):



C. Scattered

☐ bipartite ☐ not

Evidence (a colouring, or an odd cycle):



D. The 8-node graph

☐ bipartite ☐ not

Evidence (a colouring, or an odd cycle):

3.2 *Skill: bipartite and odd cycles, both directions*

Clicker · Lecture 6, slide 8

Which of the following is true?

- ◯ **A.** If G is bipartite, then G does not have an odd cycle
- ◯ **B.** If G does not have an odd cycle, then G is bipartite
- ◯ **C.** Both A and B
- ◯ **D.** Neither A nor B

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.3 *Skill: the odd cycle the proof builds*

The BFS test finds an edge (x, y) with both ends in layer 5. Their lowest common ancestor in the BFS tree is in layer 2. How many edges does the cycle it builds have?

Edges in the cycle = ☐

Working:

3.4 *Skill: the other half of the correctness proof*

The test finds no edge with both ends in the same layer, and colours even layers blue and odd layers orange. Why is that a valid colouring?

- ◯ **A.** Every edge joins layers that differ by exactly one, and neighbouring layers have different colours.
- ◯ **B.** BFS tree edges always join different layers.
- ◯ **C.** Because the graph has no cycles.
- ◯ **D.** Because there are only two colours.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.5 *Skill: recognising bipartite structure in a problem*

Draw a node for every student and every college, and an edge for every application. Why is this graph always bipartite?

- ◯ **A.** Every edge joins a student to a college, so students one colour and colleges the other is a colouring.
- ◯ **B.** Because every student applies to the same number of colleges.
- ◯ **C.** Because stable matchings always exist.
- ◯ **D.** It is not always bipartite.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.6 *Skill: recognising the two-colour test in a problem that does not say "graph"*

There are n professional wrestlers and a list of r pairs of them who are rivals. Can each wrestler be made a "babyface" (good guy) or a "heel" (bad guy) so that every rivalry is between a babyface and a heel? Which of these answers it, and gives the split, in O(n + r) time?

- ◯ **A.** Run the BFS two-colour test once, from any wrestler.
- ◯ **B.** Make a node for each wrestler and an edge for each rivalry. While some wrestler has no colour, run the BFS two-colour test from them. Blue are babyfaces, orange are heels.
- ◯ **C.** Try every way of splitting them, 2^n in all.
- ◯ **D.** Run Gale–Shapley, with babyfaces proposing to heels.

Why not the others? Pick the one you almost chose and say what is wrong with it.

All edges forward

In a **directed** graph each edge (u, v) points from u to v. A path has to follow the arrows, so v can be reachable from u without u being reachable from v. BFS works unchanged, following only edges that leave a node. To find everything that can reach t, run BFS on the graph with every edge turned round.

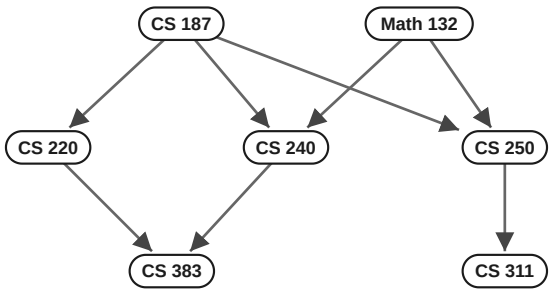
A **DAG** is a directed graph with no directed cycle. A **topological order** lists the nodes so that every edge points forward, from earlier to later.

The sort

1. Find a node with no incoming edges (a source).
2. Put it next in the order.
3. Delete it and its outgoing edges. Repeat until nothing is left.

Why it works. Every DAG has a source: walk backwards along incoming edges, and without a source the walk would go on for ever and repeat a node, which is a cycle. Deleting a source leaves a smaller DAG, so by induction the sort never gets stuck on a DAG. And a graph with a cycle has no topological order at all, since a cycle has to come back. **So G has a topological order exactly when it is a DAG.**

Worked example: course prerequisites

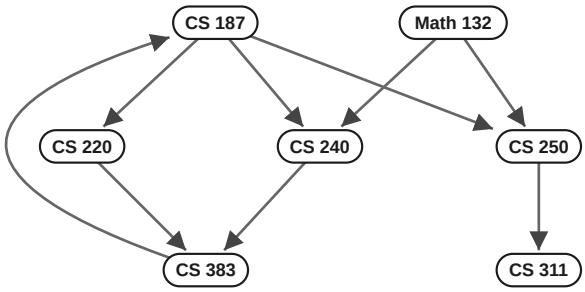


Always taking the first source available gives Math 132, CS 187, CS 220, CS 240, CS 250, CS 311, CS 383. Choosing differently when there is a choice gives other orders, and all 46 of them are right.

In CLRS. §22.4 sorts with DFS instead: run it over the whole graph and list the nodes in decreasing finishing time. On these courses that gives CS 187, CS 220, Math 132, CS 250, CS 311, CS 240, CS 383, another of the valid orders. The sort above is CLRS Exercise 22.4-5.

Beyond DAGs. A **strongly connected component** is a largest set of nodes that can all reach each other. Squash each to one node and what is left is always a DAG. Tarjan (1972) finds them in O(m + n).

WARM-UP W4 · ONE BAD RULE



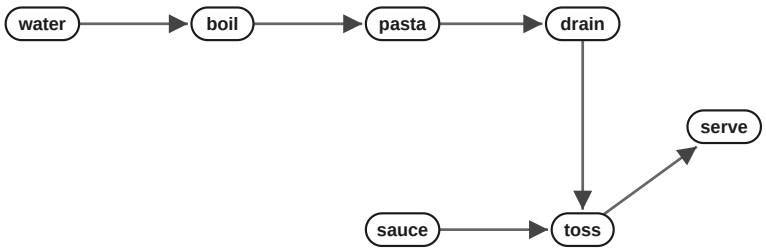
Someone adds the rule "CS 383 before CS 187". Run the sort. Where does it get stuck, and what is the cycle that stops it?

PART 4 · QUESTIONS

Tier 4: All edges forward

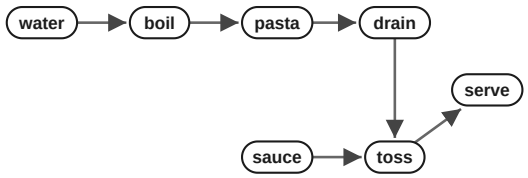
4.1 *Skill: producing a topological order*

Put the jobs in an order where every job comes after everything it waits for.



1	2	3	4	5	6	7

4.2 *Skill: how much freedom a topological order has*



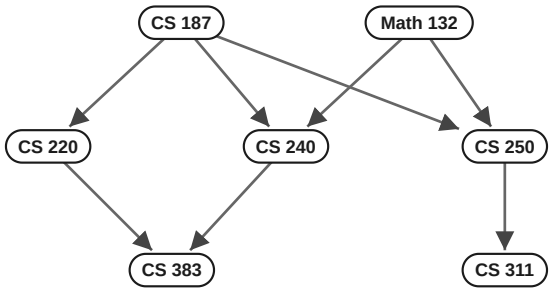
How many different valid orders are there for dinner?

Valid orders = ☐

Working:

4.3 *Skill: checking an order against every edge*

Which of these course orders are topological? Tick every valid one.



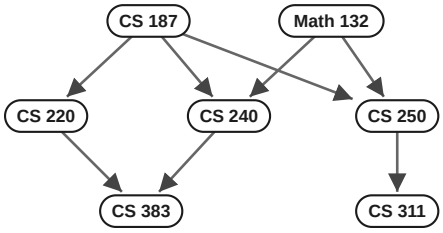
A ☐ valid `Math 132, CS 187, CS 220, CS 240, CS 250, CS 311, CS 383`
If not, which arrow points backward? _____

B ☐ valid `CS 187, CS 220, Math 132, CS 250, CS 311, CS 240, CS 383`
If not, which arrow points backward? _____

C ☐ valid `CS 187, CS 220, CS 383, Math 132, CS 240, CS 250, CS 311`
If not, which arrow points backward? _____

D ☐ valid `Math 132, CS 187, CS 250, CS 311, CS 240, CS 220, CS 383`
If not, which arrow points backward? _____

4.4 *Skill: CLRS's topological sort, by finishing time*



CLRS sorts a DAG another way: run DFS over the whole graph, starting a new search at each unexplored node in turn, and as each call returns put its node at the front of a list. Run it on the courses, taking nodes and neighbours in the order Math 132, CS 187, CS 220, CS 240, CS 250, CS 311, CS 383. What comes out?

- ◯ **A.** Math 132, CS 187, CS 220, CS 240, CS 250, CS 311, CS 383, the same as the page's sort, since a DAG has only one right order.
- ◯ **B.** CS 187, CS 220, Math 132, CS 250, CS 311, CS 240, CS 383: a different order from the page's, and just as valid.
- ◯ **C.** CS 383, CS 240, CS 311, CS 250, Math 132, CS 220, CS 187: the courses in the order their calls return.
- ◯ **D.** Nothing: DFS needs a single start, and two courses have no prerequisites.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.5 *Skill: what a topological order says about the number of edges*

Clicker · Lecture 6, slide 22

What is the maximum number of edges in a DAG with 6 nodes?

Most edges = ☐

Working:

4.6 *Skill: directed reachability, and why direction matters*

Clicker · Lecture 6, slide 14



G is a directed path on 5 vertices. BFS is called repeatedly, starting from any unexplored vertex, until all nodes are explored. What is the maximum number of times BFS may be called?

Calls, at most = ☐

Working:

4.7 *Skill: strong components, and the DAG they make*

Clicker · Lecture 6, slide 24

Consider the graph G′ whose nodes are the strong components, with an edge from C to D if any node in C has an edge to any node in D. Which of the following is always true?

- ◯ **A.** G′ is strongly connected
- ◯ **B.** G′ has a cycle
- ◯ **C.** G′ has at least n/2 nodes
- ◯ **D.** G′ is a DAG

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.8 *Skill: the step the topological sort proof needs*

Why does every DAG have a node with no incoming edges?

- ◯ **A.** Walk backwards along incoming edges. If none of those nodes were a source the walk would never stop, and with only n nodes it would repeat one, which gives a cycle.
- ◯ **B.** Because a DAG has fewer than n(n - 1)/2 edges.
- ◯ **C.** Because the first node in the topological order is a source.
- ◯ **D.** Because every node has at least one outgoing edge.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.9 *Skill: making the source-removal sort O(m + n)*

CLRS Exercise 22.4-5 asks for the page's sort, repeatedly taking a node with no incoming edges and deleting it, in O(m + n) time. What makes it that fast?

- ◯ **A.** Before each step, scan every node left for one with no incoming edges.
- ◯ **B.** Sort the nodes once by how many incoming edges they start with, and place them in that order.
- ◯ **C.** Nothing can: just finding a source takes O(n), so n steps cost $O(n^2)$.
- ◯ **D.** Count each node's incoming edges once. Keep the nodes whose count is 0 in a list. Placing a node subtracts one from the count of each node it points to, and any count that reaches 0 joins the list.

Why not the others? Pick the one you almost chose and say what is wrong with it.

KEEP THIS PAGE

One-page summary

Words

- Path: every step an edge. Simple: no repeated node. Cycle: back to the start, nothing else repeated, at least 3 nodes.
- Distance: fewest edges on a path. Connected component: largest set with paths between all pairs.
- Tree: connected, no cycles; $n - 1$ edges.

BFS

- Layer i = nodes at distance i. Order changes the tree, never the layers.
- An edge joins the same layer or neighbouring layers, never further.
- $O(m + n)$: inner lines run $2m$ times, the rest at most n.

DFS

- Recursive; returns only when all its node's edges are looked at.
- Non-tree edges join a node to an ancestor.
- $O(m + n)$ over all calls.

Generic traversal

- A queue $\rightarrow$ BFS. A stack $\rightarrow$ DFS.
- Every neighbour goes into A; $w \neq s$ goes in $\deg(w)$ times when all is explored.
- Loop over unexplored nodes $\rightarrow$ one search per component, $O(m + n)$ total.

Bipartite

- Two sides, every edge across. Bipartite $\Leftrightarrow$ no odd cycle.
- Test: BFS, colour by layer parity, look for a same-layer edge.
- Same-layer edge in layer j, lowest common ancestor in layer i $\rightarrow$ odd cycle of length $2(j - i) + 1$.

Directed graphs

- Reachability is one-way. Who can reach t: BFS on the reversed graph.
- DAG: no directed cycle. Topological order: every edge forward.
- Has a topological order $\Leftrightarrow$ is a DAG.
- Sort: take a source, delete it, repeat. Any source is fine. Stuck $\Leftrightarrow$ cycle. (CLRS: decreasing DFS finishing time.)
- A DAG on n nodes has at most $n(n - 1)/2$ edges.
- Strong components squash to a DAG.

Proof moves

- Count how often each line runs.
- "Whichever end is discovered first."
- Take the least common ancestor.
- Walk backwards until something repeats.

ANSWER KEY

Answer key: warm-ups

Try everything before you look.

W1 · BFS from SRI

L_0	L_1	L_2	L_3	L_4	L_5
SRI	UTAH UCSB STAN UCLA	MIT SDC RAND	BBN LINC	HARV CASE	CARN

Tree edges: SRI–UTAH, SRI–UCSB, SRI–STAN, SRI–UCLA, UTAH–MIT, UTAH–SDC, UCLA–RAND, MIT–BBN, MIT–LINC, BBN–HARV, LINC–CASE, HARV–CARN. Not in the tree: BBN–RAND, SDC–RAND, CASE–CARN, STAN–UCLA, UCSB–UCLA.

Same graph as the worked example, different start, so different layers: every layer is a distance *from the start*. Still 12 tree edges and 5 others.

W1b · counting lines

line	runs
`mark s discovered; L[0] = {s}; i = 0`	1
`while L[i] is not empty`	5
`L[i+1] = empty list`	4
`for each node v in L[i]`	13
`for each edge (v, w)`	34
`if w is not discovered`	34
`mark w; add w to L[i+1]; add (v, w) to T`	12
`i = i + 1`	4

n = 13, m = 17. The two inner lines run 2m = 34 times; every other line at most n times. Total O(m + n).

W2 · the generic traversal from 1

A = queue: explored in the order 1, 2, 3, 4, 5, 7, 8, 6. Tree: 1–2, 1–3, 2–4, 2–5, 3–7, 3–8, 5–6. That is BFS order: 1, then its neighbours, then theirs.

A = stack: explored in the order 1, 3, 8, 7, 5, 6, 4, 2. Tree: 1–3, 3–8, 8–7, 3–5, 5–6, 5–4, 4–2. Depth first, though not the same tree as recursive DFS: the stack hands neighbours back newest first.

Times each node went into A (same either way): 1: 3, 2: 4, 3: 5, 4: 2, 5: 4, 6: 1, 7: 2, 8: 2. Every node except 1 goes in exactly degree-many times; 1 once more, at the start.

W3 · the cube

Bipartite. One side: a, c, f, h. Other side: b, d, e, g. BFS from a: layers {a}, {b, d, e}, {c, f, h}, {g}, and no edge joins two nodes in one layer. It has plenty of cycles, of length 4, 6 and 8, and every one is even.

W4 · the bad rule

Only Math 132 can be placed. After that every course left has an arrow pointing in from another course left, so there is no source and the sort is stuck. The cycle: CS 220 → CS 383 → CS 187 → CS 220. No order can put each of these before the next, since the last has to come before the first.

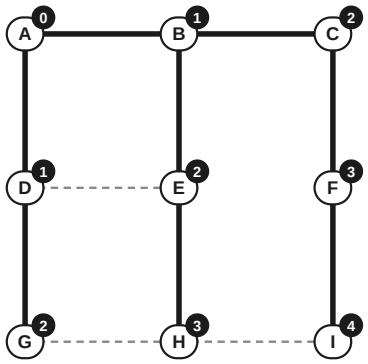
ANSWER KEY

Answer key: questions

1.1

Layers from A:

L_0	L_1	L_2	L_3	L_4
A	B D	C E G	F H	I



Solid: one BFS tree. Any node's layer is its distance, so check each by finding a shortest path.

Takeaway. A layer is a distance, nothing more: layer i is every node whose shortest path from A has exactly i edges. You can check any answer without running anything, by finding a shortest path.

1.2

Answer: **Nodes in layer 2 = 5**

Layer 1 is MIT's neighbours: UTAH, BBN and LINC. Layer 2 is everything new next to those: SRI and SDC from UTAH, RAND and HARV from BBN, and CASE from LINC.

- **If you wrote 4:** CASE is missing. LINC is in layer 1 as much as UTAH and BBN are, and CASE is new when LINC is explored.
- **If you wrote 3:** That is layer 1: the nodes one edge from MIT. Layer 2 is two edges away.
- **If you wrote 6:** Something is counted twice or too early. SDC and RAND are next to each other but each is in layer 2 only once, and UCLA is not two edges from MIT.

Takeaway. Layer 2 is exactly the nodes at distance 2. RAND is next to SDC, another layer-2 node, and that edge changes nothing: an edge inside a layer is allowed, it is just not a tree edge.

1.3

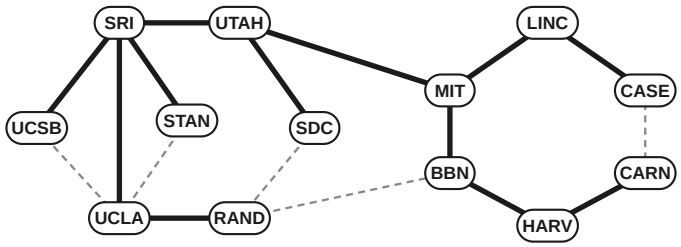
- **P – Q – R – S**: Simple path. Every step is an edge and no node repeats.
- **S – T – U – S**: Cycle. Back to S, and nothing else repeats.
- **R – S – R – P**: Path, not simple. R comes up twice.
- **Q – R – S – U – V**: Simple path. Every step is an edge and no node repeats.
- **P – R – T**: Not a path. R–T is not an edge.
- **P – Q – R – P**: Cycle. Back to P, and nothing else repeats.

- **U – S – T – U – V**: Path, not simple. U comes up twice.

Takeaway. A path only needs every step to be an edge. Simple forbids repeating a node. A cycle ends where it began and repeats nothing else. Those are three different checks, in that order.

1.4

Not in the tree: **BBN–RAND, SDC–RAND, CASE–CARN, STAN–UCLA, UCSB–UCLA**



Solid: tree edges. Dashed: the rest.

Takeaway. A tree on n nodes has n - 1 edges, so 12 of the 17 are tree edges and exactly 5 are not, whatever the start. Which 5 it is depends on the start and on the order neighbours are looked at.

1.5

Answer: **Times it runs = 22**

It runs once for every (v, w) pair looked at, and every edge is looked at from both ends: 11 edges, twice each. That is where the 2m in O(m + n) comes from.

- **If you wrote 11:** That is m. Each edge is looked at twice, once when each end is explored.
- **If you wrote 8:** That is how often the test succeeds, once for every node except A. The test runs whether it succeeds or not.
- **If you wrote 9:** That is n, how often "for each node v in L[i]" runs.

Takeaway. The running-time proof is exactly this count, done in general: each line runs at most n times or at most 2m times, so the whole thing is O(m + n).

1.6

Answer: **B**

- **A** × The 2m in that bound counts neighbours handed over one at a time by a list. A row has n entries whatever the degree of v, and every one has to be read.
- **B** ✓ Each node's row is read once, when that node is explored: n rows of n entries. On the 1970 Internet that is 169 entries read, against 34 entries in all the adjacency lists put together.
- **C** × That would be reading a whole row once per edge. Each row is read once, when its node is explored.
- **D** × That is one row. BFS reads a row for every node it explores.

Takeaway. This is CLRS Exercise 22.2-4. CLRS §22.1 makes the same trade for storage: lists take $\Theta(m + n)$ space and a matrix $\Theta(n^2)$ whatever m is. (CLRS writes $\Theta(V + E)$ and $\Theta(V^2)$, with V and E standing for n and m.) On a sparse graph like the 1970 Internet the lists win. On a dense one m is close to n^2 and the two come to the same.

1.7

Answer: **B**

- **A** × Layer i is the set of nodes at distance i from s. Distance does not depend on anybody's order.
- **B** ✓ A node with two neighbours in the layer above gets whichever is explored first as its parent. On the 1970 Internet UCLA can hang from SRI or from RAND.
- **C** × The tree can change, but the layers cannot: they are distances.
- **D** × The layers stay put, but a node can be found from a different parent. Try it on the page with the order reversed.

Takeaway. Layers are a property of the graph. The tree is a property of the run. Anything you prove about BFS layers holds however the run is ordered. CLRS Exercise 22.2-5 asks for exactly this: the distances BFS finds do not depend on the order, and the tree can.

1.8

Answer: **A**

- **A** ✓ That is the proof. Either y was already discovered, in layer i or i + 1, or it is discovered right then, into i + 1.
- **B** × True, but it says nothing about y in particular. The argument has to use the edge between x and y.
- **C** × Connectivity is not used. The claim holds for any edge in any graph, as long as BFS reaches it.
- **D** × The setup says x was discovered first. Choosing which end to call x is the "without loss of generality" step.

Takeaway. This fact does real work later: the bipartite test in chapter 3 rests on it. An edge joins the same layer or neighbouring layers, and nothing else. CLRS states it as Lemma 22.1, $\delta(s, v) \leq \delta(s, u) + 1$ for every edge (u, v), and in an undirected graph that holds both ways round.

1.9

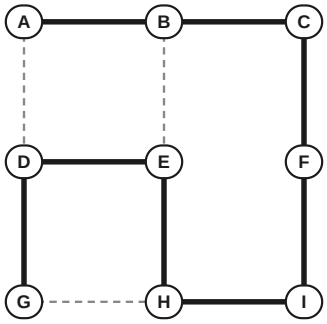
Answer: **B**

- **A** × True. Every edge is on a cycle, so there is always another way round. Even UTAH–MIT has the route through RAND and BBN.
- **B** ✓ False, so this is the answer. Deleting SRI–UCSB and SRI–STAN, for one, leaves everything connected.
- **C** × True. No single site is the only way between two others.
- **D** × True. Deleting MIT and BBN cuts the west coast off from the east. Deleting UCSB and STAN leaves it connected.

Takeaway. "Must" claims fall to one counterexample, "there is a way" claims to one example. Most of the work is noticing which kind a statement is.

2.1

Tree edges: **A–B, B–C, C–F, F–I, I–H, H–E, E–D, D–G**



Solid: tree edges. Dashed: the rest.

Takeaway. DFS goes as deep as it can before it backs up, so its tree is long and thin where BFS's is short and wide. Both have n - 1 edges on a connected graph.

2.2

Answer: **S.d = 6**

R starts at 1 and calls P at 2, which calls Q at 3. Q finds nothing new and returns at 4, P returns at 5, and only then does the loop in DFS(R) reach S, at 6.

- **If you wrote 4:** That is the place of S in the order DFS explores nodes: R, P, Q, S. The clock also ticks when a call returns, and DFS(Q) and DFS(P) both return before DFS(S) starts.
- **If you wrote 13:** That is S.f, when DFS(S) returns, after T, U and V have each been started and finished inside it.
- **If you wrote 5:** That is P.f, when DFS(P) returns. S starts on the next tick.

Takeaway. A node's two stamps bracket the stamps of everything below it: S is 6/13, and T, U and V all fall inside. P is 2/5, and P and S never overlap, since neither is below the other. That is CLRS's parenthesis theorem (Theorem 22.7): two intervals are nested or apart, never half overlapping. The lectures do not timestamp, but CLRS builds its topological sort on u.f.

2.3

Answer: **D**

- **A** × True. Every edge adds one to the degree of each of its two ends.
- **B** × True. That is what "average" means: the sum divided by n.
- **C** × True. q = 2m, and when m ≥ n, m ≤ m + n ≤ 2m, so 2m is Θ(m + n).
- **D** ✓ All three are true, so none is false.

Takeaway. This is why "for each edge (v, w)" costs O(m) over a whole search, not O(n) per node: the total is the degree sum, 2m, and the adjacency list delivers each neighbour in constant time.

2.4

Answer: **D**

- **A** × True. A node is only marked when it is not yet marked, and the question says every node gets marked.
- **B** × True. Every neighbour goes into A, explored or not, so a node with three neighbours can go in three times.
- **C** × True. w goes in once each time one of its neighbours is explored, and every node is explored exactly once. (s goes in one extra time, at the start.)
- **D** ✓ False. Everything explored was reached from s, so every node has a path to s, and x to y goes via s.

Takeaway. The page's traversal counts every put. Run it on the 8-node graph and check C: each node other than the start goes in exactly as many times as it has edges. CLRS's BFS (§22.2) marks a node gray as it goes into the queue instead, so there nothing goes in twice and B would be false. The code here marks on the way out, which is what lets the same code be DFS with a stack.

2.5

Answer: **Searches started = 4**

One search per connected component, since a search explores its whole component and nothing else. The pieces are {1, 5, 9}, {2, 3, 6}, {4, 8} and {7} on its own.

- **If you wrote 3:** 7 is missing. A node with no edges is a component by itself, and the loop has to start a search there too.
- **If you wrote 9:** That is one per node. A search explores a whole component, so the nodes it reaches are never started from.
- **If you wrote 1:** The graph is not connected: no edge joins 1, 5 or 9 to anything else.

Takeaway. Finding every component is still O(m + n) in total. Each search costs time for its own component's nodes and edges, and the components share none. CLRS's DFS always runs this loop, and its Exercise 22.3-12 is this question: the depth-first forest has one tree per component.

2.6

Answer: **B**

- **A** × No. Either way the search explores exactly what is reachable from s.
- **B** ✓ A queue finishes a layer before starting the next. A stack follows the newest node first and goes deep.
- **C** × Each put and take is O(1) for a queue and for a stack, and the count of puts is the same 2m + 1.
- **D** × The pseudocode leaves open which node "take" takes. That choice is the difference.

Takeaway. The whole difference between breadth first and depth first is which end of A you take from.

2.7

Answer: **C**

- **A** × Leaving out the duplicates changes who gets to be a parent. The page's version lets the newest copy of a node win. This one fixes the parent at the first put, which is the oldest.
- **B** × It explores exactly what 1 can reach, like every version. What goes wrong is the shape of the tree.
- **C** ✓ In a depth-first tree that cannot happen: whichever of 2 and 3 is explored first would find the other inside its own call. CLRS Theorem 22.10 says a DFS of an undirected graph has only tree edges and back edges, and 2–3 is neither.
- **D** × The stack only decides which node is taken next. Marking on the way in fixes each parent too early, before the search has gone deep.

Takeaway. Where the mark goes matters. CLRS's BFS marks a node gray as it joins the queue, and for a queue that is fine. CLRS Exercise 22.3-7 asks for DFS with a stack instead of recursion, and the safe way is the page's: mark a node when it comes out. The page's stack version is then recursive DFS with every neighbour list read back to front.

2.8

Answer: **A**

- **A** ✓ Say x is explored first. DFS(x) looks at the edge (x, y). If y were still unexplored it would become x's child, so y was explored during DFS(x), which makes it a descendant of x.
- **B** × That happens with BFS, where UCSB–UCLA joins two children of SRI. In DFS it cannot: whichever is explored first finds the other before its call returns.

- **C** × Only possible if they were in different branches, and they cannot be.
- **D** × They have exactly the structure in the first option, and that is what makes DFS useful for finding cycles.

Takeaway. BFS non-tree edges run sideways, within a layer or to the next one. DFS non-tree edges run up and down one branch. Same graph, opposite shapes. CLRS calls these back edges, and its Theorem 22.10 is this claim: in a DFS of an undirected graph every edge is a tree edge or a back edge.

3.1

- **A. A small town**: bipartite. One side: A, C, E, G, I; the other: B, D, F, H.
- **B. Two triangles**: not bipartite. Odd cycle of length 3: P – Q – R – P.
- **C. Scattered**: not bipartite. Odd cycle of length 3: 2 – 6 – 3 – 2.
- **D. The 8-node graph**: not bipartite. Odd cycle of length 3: 1 – 2 – 3 – 1.

Takeaway. To show a graph is bipartite, give the colouring. To show it is not, give an odd cycle. Either one is short enough to check by eye, which is what makes this a good certificate.

3.2

Answer: **C**

- **A** × True, but so is B. Going round a cycle switches colour at every step, so getting back to the start takes an even number of steps.
- **B** × True, but so is A. This is the hard direction, and the BFS test proves it: if the test says "not bipartite", it has found an odd cycle.
- **C** ✓ Bipartite exactly when there is no odd cycle.
- **D** × Both are true. A is the easy claim, B is what the algorithm proves.

Takeaway. The algorithm proves the theorem. When BFS finds a same-layer edge, the odd cycle through the lowest common ancestor is the proof that no colouring exists.

3.3

Answer: **Edges in the cycle = 7**

From the ancestor down to x is 5 - 2 = 3 edges, the same again down to y, and 1 more for the edge (x, y): 2(j - i) + 1 = 7.

- **If you wrote 6:** That leaves out the edge (x, y) itself, which closes the cycle.
- **If you wrote 11:** That goes all the way up to the root. The two paths meet at layer 2, not layer 0.
- **If you wrote 3:** That is one tree path. There are two, one to x and one to y.

Takeaway. Whatever j and i are, 2(j - i) + 1 is odd. That one line is why a single same-layer edge settles the question.

3.4

Answer: **A**

- **A** ✓ BFS allows a difference of 0 or 1, and the test ruled out 0.
- **B** × True, but the colouring has to work for every edge, including the non-tree ones.
- **C** × It can have plenty of cycles, as long as they are even. The cube does.
- **D** × Two colours is the goal, not the reason it succeeds.

Takeaway. Both halves of the proof lean on the lecture 5 fact: an edge joins the same layer or neighbouring ones. Same layer means an odd cycle, neighbouring layers means different colours.

3.5

Answer: **A**

- **A** ✓ The two kinds of thing are the two sides. No edge ever joins two students or two colleges.
- **B** × Numbers do not matter. What matters is that no edge joins two things of the same kind.
- **C** × That is a fact about preferences. Bipartiteness is about which pairs an edge can join.
- **D** × It always is, for the reason in the first option.

Takeaway. Plenty of problems come with the two sides built in: students and colleges, clients and servers, jobs and machines. Then the colouring is free.

3.6

Answer: **B**

- **A** × Nothing says every wrestler is linked to every other through rivalries. One search only colours its own piece, and wrestlers in the other pieces are never reached.
- **B** ✓ The question is exactly "is this graph bipartite", on n nodes and r edges. Each search pays only for its own piece, so together they are O(n + r), and a same-layer edge in any piece means the answer is no.
- **C** × Correct, but exponential. The test never guesses: once one wrestler in a piece is fixed, every rival's side is forced.
- **D** × Who is a babyface is what has to be found, not something given, and nobody has preferences. This is a colouring problem, not a matching one.

Takeaway. This is CLRS Exercise 22.2-7. The two sides are not handed over, the way students and colleges were. The test finds them, or finds an odd cycle of feuds (three wrestlers who all feud with one another, say), and no split can satisfy that.

4.1

Any valid order is right: there are **5**. One of them: Fill the pot, Boil it, Make sauce, Cook pasta, Drain, Toss together, Serve.

To mark yours, check every arrow: Fill the pot before Boil it; Boil it before Cook pasta; Cook pasta before Drain; Drain before Toss together; Make sauce before Toss together; Toss together before Serve.

Takeaway. Always take something nothing is waiting on. It exists as long as what is left has no cycle, and taking it can never block anything else.

4.2

Answer: **Valid orders = 5**

Everything except the sauce is one fixed chain: fill, boil, cook, drain, toss, serve. The sauce only has to come before tossing, so it can go before any of the first five jobs in the chain.

- **If you wrote 1:** The sauce is free to move. Nothing points into it, and it only has to beat "toss together".
- **If you wrote 7:** The sauce cannot come after tossing or serving, since tossing needs it.
- **If you wrote 6:** Count the gaps before "toss together": before fill, boil, cook and drain, and just before toss. That is 5.

Takeaway. A DAG usually has many topological orders. The algorithm finds one, and any of them is right. So the question to ask of an answer is "is it valid", never "is it the one".

4.3

Valid: **A, B, D**

- **A** ✓ every arrow points forward.
- **B** ✓ every arrow points forward.
- **C** × CS 240 → CS 383 points backward.
- **D** ✓ every arrow points forward.

Takeaway. To check an order, look at every edge and make sure it points forward. One edge pointing back is enough to rule it out, and naming that edge is the whole explanation.

4.4

Answer: **B**

- **A** × These courses have 46 valid orders, and the two methods need not land on the same one. Math 132 is started first, but its call returns before CS 187 is even started.
- **B** ✓ When DFS looks along an edge (u, v), v is either new, so it finishes inside the call for u, or finished already. It cannot still be running, since that would make a cycle. So every course finishes before anything that leads to it, and the last to finish goes first.
- **C** × That is backwards. CS 383 finishes first because no arrow leaves it, so it belongs at the end. Each finished course goes on the front of the list, and the last one to finish ends up first.
- **D** × CLRS's DFS loops over every node and starts a new tree at any it has not reached, the same loop that finds every component in chapter 2.

Takeaway. This is CLRS's TOPOLOGICAL-SORT (§22.4, Theorem 22.12). It rests on Lemma 22.11: a directed graph is a DAG exactly when DFS finds no back edge, no edge into a call that is still running. The page's sort, always taking a node nothing points into, is the lectures' own and CLRS Exercise 22.4-5. Both are O(m + n), and both are right.

4.5

Answer: **Most edges = 15**

Put the nodes in topological order. Every edge points from an earlier node to a later one, so there is at most one per pair: $6 \times 5 / 2 = 15$. Having every forward edge reaches that.

- **If you wrote 30:** That is every ordered pair, n(n - 1). Both u → v and v → u would make a 2-cycle.
- **If you wrote 5:** That is a single path. A DAG can have far more, like 1 → 3 as well as 1 → 2 → 3.
- **If you wrote 36:** That is n^2. A DAG has no edges from a node to itself and no pair in both directions.

Takeaway. In general n(n - 1)/2, which is $\Theta(n^2)$. So on a DAG m can be as big as n^2, and O(m + n) can mean $O(n^2)$.

4.6

Answer: **Calls, at most = 5**

Start at the far end, v5: it reaches nothing new. Then v4 reaches only v5, which is already explored, and so on back to v1. Every call explores one node.

- **If you wrote 1:** That is the fewest, starting at v1. The question asks for the most.
- **If you wrote 4:** That is n - 1, the number of edges. Every call explores at least one node, and a bad order makes it exactly one.

Takeaway. In an undirected graph the number of searches is the number of components, whatever the order. In a directed graph it depends on where you start, because reaching is one-way.

4.7

Answer: **D**

- **A** × Only when there is one component. A one-way edge between two components cannot be walked back.

- **B** × The opposite. A cycle through C and D would let every node in C reach every node in D and back, so they would be one component.
- **C** × A strongly connected graph has one component, however big n is.
- **D** ✓ Any cycle in G′ would merge the components on it into one, which contradicts their being maximal.

Takeaway. Every directed graph is a DAG of strongly connected pieces. Knowing that shape is often most of an algorithm. CLRS Lemma 22.13 is the same argument, and CLRS §22.5 finds the pieces with two depth-first searches, one on G and one on G with every edge turned round.

4.8

Answer: **A**

- **A** ✓ That is the proof. A finite graph plus "keep going forever" means a repeat, and a repeat means a cycle.
- **B** × The edge count does not rule anything out: a DAG can have exactly n(n - 1)/2 edges.
- **C** × That assumes an order exists, which is what this fact is used to prove. Circular.
- **D** × Not true of DAGs: the last node in any order has none.

Takeaway. With this fact the sort cannot get stuck on a DAG. Take a source, delete it, and what is left is still a DAG, so it has a source too. That is the induction.

4.9

Answer: **D**

- **A** × Each scan has to count incoming edges again, up to O(m + n) a time, and there are n steps.
- **B** × The counts change as nodes are deleted, so the starting ones mislead. CS 311 starts with 1 incoming edge and CS 250 with 2, so CS 311 would go first, but CS 250 → CS 311 says it comes after.
- **C** × Finding a source is O(1) if sources are kept in a list as they appear. The work then goes into keeping the counts up to date, and that is one subtraction per edge.
- **D** ✓ Counting is O(m + n). Each edge is subtracted once, when the node it leaves is placed, and each node joins and leaves the list once. O(m + n) in all.

Takeaway. If there is a cycle, the list runs dry while nodes are left, which is "stuck" in chapter 4, and the nodes left over contain the cycle. CLRS asks about that case in the same exercise. The page's own code just rescans, since its graphs are small.